

[illegible]

例 计算器的属性文法 (续)

产生式	语法规则	为变量附加综合属性	遇到 * : $\text{父.val} = \text{左.val} * \text{右.val}$
$E \rightarrow E + T$	$\text{print}(E.\text{val})$	$E.\text{val} = E_1.\text{val} + T.\text{val}$	遇到 + : $\text{父.val} = \text{左.val} + \text{右.val}$
$E \rightarrow E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$	$E.\text{val} = E_1.\text{val} + T.\text{val}$	遇到单符 : $\text{父.val} = \text{子.val}$
$T \rightarrow T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$	$T.\text{val} = T_1.\text{val} * F.\text{val}$	遇到括号 : $\text{父.val} = \text{括号内.val}$
$T \rightarrow F$	$T.\text{val} = F.\text{val}$	$T.\text{val} = F.\text{val}$	遇到 digit : $\text{父.val} = \text{digit.lexval}$
$F \rightarrow (E)$	$F.\text{val} = E.\text{val}$	$F.\text{val} = E.\text{val}$	遇到左括号 : $\text{父.val} = \text{右括号.val}$
$F \rightarrow \text{digit}$	$F.\text{val} = \text{digit.lexval}$	$F.\text{val} = \text{digit.lexval}$	遇到右括号 : $\text{父.val} = \text{digit.lexval}$
			遇到右括号和左括号同名，比如： $E \rightarrow E * T$
			语法规则里要写成： $E \rightarrow E_1 * T + T$

做法: 先列出每个文法符号的属性名和方向: 综合、继承或固有。再对每条产生式写语义规则, 规则右侧只此时可获得的属性。综合属性常由子结点向上算, 继承属性常由父结点和左兄弟向右传。

第 6 章 | 注释分析树

语义计算

还翻译表达式中的数量

语义规则

```

print (E.val)
E.val → E1.val + T.val
E → T
T → T1*F
T → T1F
T.val → T1.val * F.val
F → digit
F.val → digit.lexval

```

$E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow T_1 * F$
 $T \rightarrow T_1 F$
 $F \rightarrow \text{digit}$

$E.val = 19$
 $T.val = 4$
 $F.val = 4$
 $T.val = 15$
 $F.val = 5$
 $\text{digit.lexval} = 4$
 $T.val = 3$
 $F.val = 3$
 $\text{digit.lexval} = 3$

结点带属性值

注释分析树带注释法

只能进行语义的运算操作
E管 + / 乘
F管 digit / 指

从 L 开始，
运算符 + 构建
(树根)

1. 数字先统一 digit, 直写 L
2. 先找最外层
3. 用 E → ...
4. 5. 括弧用 F → ...
6. 单个数字 F → ...
7. F → digit
8. F → digit

做法：先画普通语法树，再把每个结点的属性值写在结点旁。按依赖关系求解：叶子固有属性先给，综合属性上，推求属属性父结点或左兄弟传入。若表格难画，说明该题求值次序非法。

第6章 | L 属性定义

其属性可用深度优先的顺序从左至右计算

- 对于所有 $A \Rightarrow X_1, X_2, \dots, X_n$
- X_n 属性计算仅使用 $X_1, X_2, \dots, X_{n-1}$ 的属性和 A 的继承属性

S-属性文法为特殊的 L-属性文法

$Fval := digit(L)$
 适合自底向上计
 常值归约的计算

L-属性文法：能继承但可能依赖
 1. 左部 A 的继承属性
 2. X_i 左边兄弟 $X_{i-1}, X_{i-2}, \dots$ 的属性
 3. X_i 自己的继承属性
 继承属性只能从左至右传递，不能从右至左传递。

合法 L 属性：
 $XLin := A$
 $XLin := X$
 $XLin := X_1 X_2 \dots X_n$
 $XLin := X_1 A$
 $XLin := A X_n$

做法: 检查每个产生式 $A \rightarrow X_1 \dots X_n$ 中 X_i 的继承属性: 只能依赖 A 的继承属性, $X_1 \dots X_{i-1}$ 的属性和 X_i 自身性。若全部满足就是 L 属性定义; 若只有综合属性则是 S 属性, 也是 L 属性特例。

第 6 章 | 翻译模式——语法制导翻译方案

• 将语义动作中继承属性的计算前移，使它出现在其相应文法符号之前	对于 T → real, 记录 T.type = real
D → T { L.in := T.type } L T → int { T.type := integer } T → real { T.type := real } L → { L.in := L.in } L_i, id { addtype(id.entry, L.in) }	D → T { L.in := T.type } L 先分析 T 得到 T.type, 再执行动作, 把 T.type 赋给 L.in; 然后分析 L. L → id { addtype(id.entry, L.in) } 先识别 id; 再把 id 的类型赋值给 L.in. 先读取 int/real 等, 是为了花括号中给 type

做法: 把语义动作放到它所需属性已经求出的地方。若动作计算 X_i 的继承属性, 要放在 X_i 前面; 若动作计

[illegible]

例 表达式 $(A - 12) * B + 6$ 的中间代码

三地址码	四元式	三元式
$T_1 = A * 12$	$(\cdot, A, 12, T_1)$	$1(\cdot, A, 12)$
$T_2 = T_1 * B$	$(*, T_1, B, T_2)$	$2(*, (1), B)$
$T_3 = T_2 + 6$	$(+, T_2, 6, T_3)$	$3(+, (2), 6)$

波兰表示

*A12B6+

逆波兰表示

A12*B+6+

3. 画前驱图

4. 每一步生成一个临时变量

四元式：

三地址码提取
(op, arg1, arg2, result)

三元式：

四元式去掉 result，用编号引结果

波兰式（前驱式）：

画树，根左右，从左全局树根开始

逆波兰式（后驱式）：

画树，左右根，从左全局叶子开始

做法：先按表达式树求值顺序生成三地址码，再逐条转成四元式(op, arg1, arg2, result)，三元式省 result，用编号代表结果；逆波兰式是后序排列。互转时保持计算先后和临时变量引用一致。

第7章 | 逆波兰/四元式练习

3. 右边括号: $b + c$
4. 右边除法: $(b + c) / f$
5. 两边相加

[illegible]

第 7 章 | 三地址码 SDD

语法结构:
a = b + (c * d)

只	S → id := E	S.code := newEcode gen(id.place, "E.place)	即:
	E → E ₁ + E ₂	E.code := newEcode; E ₁ .code := E ₁ .code E ₂ .code gen(E ₁ .place := "E ₁ .place" + "E ₂ .place)	(c'd).place = T1 得到: c.code = gen(T1 := c * c)
-c	E → E ₁ * E ₂	E.code := newEcode; E ₁ .code := E ₁ .code E ₂ .code gen(E ₁ .place := "E ₁ .place" * "E ₂ .place)	再得: T1 := c * d
从	E → -E ₁	E.code := newEcode; E ₁ .code := E ₁ .code gen(E ₁ .place := "uminus" E ₁ .place)	再得: b + T1 申请新的临时变量: T2 := b + T1
	E → (E ₁)	E.place := E ₁ .place; E ₁ .code := E ₁ .code	最后赋值: a := T2
	E → id	E.place := id.place; E ₁ .code := "	所以完整代码: T1 := c * c * d T2 := b + T1 a := T2
	E → num	E.place := num.val; E ₁ .code := "	

第 7 章 | 数组地址计算

- 每一维的大小: $n_i = up_i - low_i + 1$, 起始地址: base
- 一维数组 $A[low_i:up_i]$

$$Addr(A[i]) = base + (i - low_i) * w = (base - low_i * w) + i * w = c + i * w$$

A[i]元素的地址在基地址+相对基地址的元素数量*每个元素的偏移
- 二维数组 $A[low_1:up_1, low_2:up_2]^T A[i_1, i_2]$

$$Addr(A[i_1, i_2]) = base + ((i_1 - low_1) * n_2 + (i_2 - low_2)) * w = base + (i_1 - low_1) * n_2 * w + (i_2 - low_2) * w = base - low_1 * n_2 * w + i_1 * n_2 * w + i_2 * w = base - flow_1 * n_2 * w + (i_1 * n_2 + i_2) * w$$

也可以写成简略形式: $c = base - (low_1 * n_2 + low_2) * w$

二维数组:
 $Addr(A[i_1, i_2]) = base + ((i_1 - low_1) * n_2 + (i_2 - low_2)) * w = base + ((i_1 - low_1) * n_2 + (i_2 - low_2)) * w$
 也可以写成简略形式:
 $c = base - (low_1 * n_2 + low_2) * w$

如果控制: 行号连续

做法：先从声明得到每维上下界和元素宽度 w 。行优先时，二维偏移 = $((i - \text{low1}) * n2 + (j - \text{low2})) * w$ ；多维依次乘后继续

[illegible]

根据 $E \rightarrow id$ (翻译 b)

产生式	语义规则	
S → id E	S.code := E.code gen(id.place, "E.place)	得到
E → E ₁ + E ₂	E.place := newtemp; E.code := E ₁ .code E ₂ .code gen(E.place := "E ₁ .place", "E ₂ .place)	b.place = b, b.code = 空 c.place = 3, c.code = 空
E → E ₁ * E ₂	E.place := newtemp; E.code := E ₁ .code E ₂ .code gen(E.place := "E ₁ .place", "E ₂ .place)	翻译 b + 3, 根据 E → E ₁ + E ₂ E.place = newtemp E.code = E1.code E2.code gen(E.place := E1.place + E2.place,
E → E ₁	E.place := newtemp; E.code := E ₁ .code gen(E.place := "uminus", E ₁ .place)	遇到负号申请: E.place = T1 , T1 = b + 3 得到: (b+3).place = T1 代入: E.code = 空 T1
E → (E ₁)	E.place := E ₁ .place; E.code := E ₁ .code	E.code = 空 T1 (b+3).code = T1 := b + 3
E → id	E.place := id.place; E.code := "	翻译 newtemp S → id := id E.code := E.code gen(id.place, "E.place")
E → num	E.place := num.val; E.code := "	S.code := T1 := b + 3 a := T1

注: 先翻译右部表达式并得到 E.place, 再按赋值目标生成最后一条代码。普通变量生成 id.place; 数组元素先按地址再生成 a[offset].place, 例如时 a[i] = b + 3, 要 b = b + 3, 普通变量 b 的 offset 是 0, 最后 a[i2] = b + 3, 要 b = b + 3, 普通变量 b 的 offset 是 i2。

第 7 章 | 赋值类型转换

```
a := b + 3;
```

<pre> → id:=E {S.code:=E.code gen(id.place):="E.place"} a:=T2 → id:=E {S.code := E.code if (id.type=E.type then gen(id.place):="E.place") else if id.type=real then gen(id.place):="rti" (E.place) else gen(id.place):="rti" (E.place)}} </pre>	先生成 E.code 再对左 id.type 和右边 E.type 类型匹配：直接赋值 int -> real: 加 rti real -> int: 加 rti 或按题目要求报错	图中 (1) (2) (5) (9) (13) (14) (23)
注：符号表表明到左值类型和右值类型。若相同直接赋值；若 int 赋 real 通常插入 rti, real 赋 int 插入 rti 或报错，取于题目规则。转换结果要放入临时变量，再用该临时变量完成赋值。	其中	6

7 章 | 拉链回滚——跳转目标暂时不知道，就先空着；等目标位置知道后，再回滚。 if a < b then S1 else

```

E.truelist = merge(E1.truelist, E2.truelist)
E.falselist = merge(E1.falselist, E2.falselist)

```

- 扫描
 - 先产生暂时没有填写目标标号的转移指令；
 - 必须对于每一条这样的指令作记录；
 - 一旦转移的目标“标号”被确定下来，再将它“回填”到相应的指令中
- 布尔表达式E设置E.true和E.falseist
 - E.true — 对应真值 **True**
 - E.falseist — 对应假值 **False**

两遍扫描

生成条件跳转时目标暂空，把这些语句编号挂到 true 假或 false 旗。控制结构入口确定后用 backpatch 统一目标；编译合并用 merge，做假时先 nextquad，再画出每条旗，最后补标号。

```

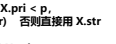
E.true = E.trueist = merge(E.trueist, E.trueist)
E.false = E.falseist = merge(E.falseist, E.falseist)
  
```

为真时跳转的位置
 $E.trueist = (2) //$
 为假时跳转的位置

每面知道 then 后入 3 条代码，就：
backpatch(E.trueist)
 于是第 1 条变成：
 1: if a < b **goto 3**

backpatch(E.falseist)
 于是第 2 条变成：
 2: **goto 8**

该文法设计语法规则定义, 目的是将后缀表达式翻译成中缀表达式(要求翻出的中缀表达式不含冗余的括号, 即: 后缀式 $ab*cd++$ 应翻译为 $a*b+(c+d)$ 而不是 $a*b+(c+d)+d$)。

例 7.1 表达式 E 的语法结构树的形式为 E.p.r: 表达式 E 的左子表达式 则左孩子: id:3 + 2 + :1	辅助函数: left(E, x, p): 若 X.pri < p, 则取 E.p: (X.str) 否则直接取 X.str	3. 删除 使用
例 7.2 E.pri = 1; E.str = left(E1, 1) + " " + right(E2, 1); → E1 * E2 E.pri = 2; E.str = left(E1, 2) + " " + right(E2, 2); → id E.pri = 3; E.str = id.lexeme;	right(X, p): 若 X.pri < p, 则取 X.p: (X.str) 否则直接取 X.str	(14) (15) 和 中 右 (14)
已生成或生成值语句的语法结构树的属性文法如下:	E.p: 指向语法树根结点的指针	
产生式 语义规则		
→ S := id → E S.p := mknode("=", mkleaf(id, id.str), E.p)		
→ E := E1 * E2 E.p := mknode("*", E1.p, E2.p)		
→ E := E1 + E2 E.p := mknode("+", E1.p, E2.p)		

$E \rightarrow (E1)$	$E.p := E1.p$	$mkleaf(id,c)$	(20)
$E \rightarrow id$	$E.p := mkleaf(id, id.entry)$	$), mknode('*',$	刪掉
		$mkleaf(id,d),$	

[illegible]

设计这个文法，要求不含 **if 结构**

While 结构

S.begin;

if 条件

goto S.begin (True, S1.begin)

goto L2 (C.False, S2.begin)

S1.begin;

goto S.next

S1.begin;

循环体代码

goto S.begin

S.next

if 条件没有 C 的情况拆开

有 C: a^n b^n c^n

无 C: a^n b^n c^+n

[S];

S-> A | A C

A-> A | A b | a

C-> C | c

if: 先翻译条件 C: C.True 回跳到 then 入口, C.False 回跳到 else 入口, 然后部分结束后若后面还有

需要 goto 则跳 else 并到 next 是, 整个 if 语句的 next 是到 循环体入口, else 出口和 then 出口

转接合用, C.False 及到 then 循环体入口, 所以为三地回到 C.True 入口

while: 记录 S 为条件测试入口, C.True 回跳到循环体入口, C.False 作为循环退出入口, 循环体入口

二地址码序列 入口：代码序列的第一语句

```

graph TD
    Start([开始]) --> IsSentenceStart{是否为句首?}
    IsSentenceStart -- 是 --> Output[输出]
    Output --> IsNextSentenceStart{是否为下一句首?}
    IsNextSentenceStart -- 是 --> IsSentenceStart
    IsNextSentenceStart -- 否 --> Continue([继续])
    Continue --> IsSentenceStart
  
```

第9章 | 程序流程图

程序流程图

块头是条件跳转：画两条边，目标边 + 顺序下一块边

块尾是无条件 goto：只画目标边（如 B5->B2）

块尾不是跳转：画顺序下一块边（如 B1->B2）

最后块没有后继就不画（如 B6）

（22）goto (5)

（12）if t5 < v goto (9)

（13）if i > j goto (23)

（5-8）

（9-12）

（13）

（14-22）

（23-30）

做法：每个基本块画成一个结点，块头如果是条件/无条件跳转，就画目标块边，执行行会顺序落到下一块，也连顺序边，检查循环时找回边；优化和活度变量分析都在图上实现代码传值信息。

公共子表达式

<pre> * 设计计算 每次一次临时变量的地方, 改成 临时变量 i t1 t2 直接: 4 * i 变量, 16 可以删掉, 直接用 j * j </pre>	<pre> * (14)(16), (17)(20), (23)(25), (26)(29) (14) t1 := 4 * i (15) x := a[t1] (16) t1 := 4 * i (17) t3 := 4 * j (18) t9 := a[t3] (19) a[t1] := t9 (20) t1 := 4 * j </pre>	<pre> (14) t1 := 4 * i (15) x := a[t1] (16) t1 := 4 * i (17) t3 := 4 * j (18) t9 := a[t3] (19) a[t1] := t9 (20) t1 := 4 * j </pre>	<pre> (23) t1 := 4 * i (24) x := a[t1] (25) t1 := 4 * i (26) t3 := 4 * n (27) t3 := a[t3] (28) a[t1] := t3 (29) t1 := 4 * n </pre>	<pre> (23) t1 := 4 * i (24) x := a[t1] (25) t1 := 4 * i (26) t3 := 4 * n (27) t3 := a[t3] (28) a[t1] := t3 (29) t1 := 4 * n </pre>
---	--	--	--	--

样, 都是 $4*j$, 所以 20 可以 (22) goto (5) (22) goto (5)
) 改成: (21) a[t3] := x